

Topology-Dependent Scaling Limits of SNN Verification via PRISM

CogSpike Research Team — March 2026

Working Paper — First Draft

Abstract. We analyze how the topology of a Spiking Neural Network (SNN) determines the state space of its corresponding DTMC (Discrete-Time Markov Chain) representation used for formal verification in PRISM. We derive closed-form expressions for state space size as a function of network structure and model configuration, validate them empirically against PRISM DTMC exports, and characterize the scaling limits of verification for different connectivity patterns. We show that discretized models achieve exponential state space reduction per neuron, and that the practical verification limit of a 4-neuron chain under the full model exceeds 362,000 states — with the precise diamond topology already exceeding PRISM’s BDD memory at 5 neurons.

1. Introduction

The formal verification of Spiking Neural Networks via probabilistic model checking is constrained by the *state space explosion problem*: the number of reachable states in the DTMC grows exponentially with network size. Our prior work [1] extended the filtration-based quotient model abstraction of Baier and Katoen [2] with a *weight discretization scheme* that preserves synaptic weight information during verification. That work introduced threshold-dependent leak, proved soundness (no spurious spikes can occur in the discretized model), and demonstrated preservation of biological properties (tonic spiking, integrator behavior, excitability) in the quotient model. The discretized model maps continuous membrane potentials to a finite set of potential classes whose size is determined by the weight level parameter W , reducing the per-neuron state domain from the full potential range to $O(W)$ classes. However, that analysis focused on the *vertical* dimension (per-neuron precision) without systematically addressing the *horizontal* dimension: how network topology (neuron count, connectivity pattern, fan-in, fan-out) determines the overall DTMC size.

This note addresses three questions from our recent research planning:

1. **DTMC Export and Inspection:** Can we export DTMCs from PRISM to observe concrete state/transition counts for different topologies?
2. **Graph-Theoretic State Space Model:** Can we derive a closed-form formula for the state space size as a function of topology parameters?
3. **Memory Estimation:** Can we predict whether a given SNN will fit within PRISM’s memory budget before attempting verification?

2. Preliminaries

We recall the CogSpike SNN model and its PRISM representation, using notation consistent with the weight discretization formalization [1].

2.1. SNN Model Recap

An SNN is a directed graph $G = (V, E)$ where:

- $V = V_{\text{in}} \cup V_{\text{proc}}$ — input neurons and processing neurons
- $E \subseteq V \times V$ — directed synaptic connections with weights $w_e \in [-100, 100]$

Each processing neuron $n \in V_{\text{proc}}$ has state variables:

- Membrane potential $p_n \in [P_{\min}, P_{\max}]$
- Spike output $y_n \in \{0, 1\}$
- Refractory state $s_n \in \{0, 1, 2\}$ (Normal, ARP, RRP) — when refractory periods are enabled
- ARP counter $\text{aref}_n \in [0, \text{ARP}]$ and RRP counter $\text{rref}_n \in [0, \text{RRP}]$

Each input neuron $i \in V_{\text{in}}$ has:

- Spike output $x_i \in \{0, 1\}$

Each internal edge $(i, j) \in E$ where $i \in V_{\text{proc}}$ has:

- Transfer variable $z_{i,j} \in \{0, 1\}$

In the **discretized quotient model** [1], the continuous membrane potentials are abstracted via a weight discretization function $\delta_{W(w)} = \lfloor w \cdot W / w_{\max} \rfloor$ that maps each synaptic weight to a finite discrete range $[-W, W]$. The discretized firing threshold $T_d = \lceil T \cdot W / w_{\max} \rceil$ is calibrated using ceiling to ensure the discretized neuron is *at least as hard* to fire as the original (Threshold Preservation Theorem [1]). The Soundness Theorem [1] further guarantees that no spurious spikes are introduced: if the original neuron should not fire, neither does the discretized one. A threshold-dependent leak factor $\lambda_d = -\max(1, \lfloor \ell \cdot T_d \rfloor)$ ensures membrane decay scales correctly with the calibrated threshold.

2.2. PRISM Module Decomposition

The PRISM model is composed of independent modules whose state spaces form a Cartesian product. CogSpike generates the following module types:

Definition. The **module decomposition** of an SNN graph $G = (V, E)$ is:

$$\mathcal{M}(G) = \{\text{GlobalClock}\} \cup \{\text{Inputs}\} \cup \{M_n : n \in V_{\text{proc}}\} \cup \{T_{i,j} : (i, j) \in E_{\text{int}}\}$$

where $E_{\text{int}} = \{(i, j) \in E : i \in V_{\text{proc}}\}$ is the set of internal (neuron-to-neuron) edges.

2.3. Model Configuration

The **ModelConfig** determines the per-neuron state complexity:

Parameter	Notation	Effect on State Space
Threshold levels	$k \in \{1, \dots, 10\}$	Determines transition fan-out
Enable ARP	$\text{arp} \in \{\text{on}, \text{off}\}$	Adds s_n and aref_n variables
Enable RRP	$\text{rrp} \in \{\text{on}, \text{off}\}$	Adds rref_n variable
Potential range	$(P_{\min}, P_{\max})$	Dominates state count per neuron
Weight levels (disc.)	$W \in \{1, \dots, 10\}$	Determines δ_W domain $[-W, W]$ and T_d ; see [1]

Table 1: Model configuration parameters and their state space impact.

3. DTMC State Space Analysis

3.1. Per-Module State Contribution

Definition. Let $|S_M|$ denote the number of possible valuations of all variables in module M . We call this the **local state space** of module M .

For each module type:

GlobalClock (present when input patterns are time-dependent):

$$|S_{\text{clock}}| = T_{\text{max}} + 1$$

Inputs (one module aggregating all input neurons):

$$|S_{\text{inputs}}| = 2^{|V_{\text{in}}|}$$

Neuron module M_n for $n \in V_{\text{proc}}$ (precise model):

$$|S_n^{\text{precise}}| = \begin{cases} 2 \cdot (P_{\text{max}} - P_{\text{min}} + 1) & \text{if arp = off} \wedge \text{rrp = off} \quad (\text{fast}) \\ (1 + \delta_{\text{arp}}) \cdot (1 + \text{ARP})^{\delta_{\text{arp}}} \cdot (1 + \delta_{\text{rrp}}) \cdot (1 + \text{RRP})^{\delta_{\text{rrp}}} \cdot 2 \cdot R_n & \text{otherwise (full)} \end{cases}$$

where $R_n = P_{\text{max},n} - P_{\text{min},n} + 1$ is the per-neuron potential range, and $\delta_{\text{arp}}, \delta_{\text{rrp}} \in \{0, 1\}$ indicate whether ARP/RRP are enabled.

For the **fast** configuration ($k = 4$, no ARP/RRP):

$$|S_n^{\text{fast}}| = 2 \cdot R_n$$

For the **full** configuration ($k = 10$, ARP + RRP with ARP=2, RRP=4):

$$|S_n^{\text{full}}| = 3 \cdot 3 \cdot 5 \cdot 2 \cdot R_n = 90 \cdot R_n$$

Neuron module M_n (discretized quotient model with weight level W):

$$|S_n^{\text{disc}}| = 2 \cdot (P_{\text{max},n}^d + 1)$$

where $P_{\text{max},n}^d = T_d + E_n^d$ with $E_n^d = \sum_{e \in \text{In}(n), w_e^d > 0} w_e^d$ being the maximum weighted contribution (in the sense of the contribution-based class evolution defined in [1]) and $T_d = \lceil P_{\text{rth}} \cdot \frac{W}{100} \rceil$ is the calibrated discretized threshold. The threshold-dependent leak $\lambda_d = -\max(1, \lfloor \ell \cdot T_d \rfloor)$ does not add new state variables but constrains the reachable potential range, ensuring the Soundness property (no spurious spikes) is maintained.

Transfer module $T_{i,j}$:

$$|S_{i,j}| = 2$$

3.2. Theoretical State Space

Theorem (State Space Product). The theoretical (upper bound) state space of the DTMC is the Cartesian product of all module state spaces:

$$|S_{\text{theory}}| = |S_{\text{clock}}| \cdot |S_{\text{inputs}}| \cdot \prod_{n \in V_{\text{proc}}} |S_n| \cdot \prod_{(i,j) \in E_{\text{int}}} 2$$

Proof. Each module’s variables are disjoint and updated only within that module (or via synchronized formulas). Since PRISM constructs the global state as a tuple of all variable valuations, the total state space is the Cartesian product of the local state spaces. $\square$

Definition. The **reachability density** ρ is the fraction of the theoretical state space that is actually reachable from the initial state:

$$\rho = \frac{|S_{\text{reachable}}|}{|S_{\text{theory}}|}$$

 **Intuition:** The reachability density ρ captures how much of the theoretical state space is “wasted.” For sparse topologies (chains), many variable combinations are unreachable because upstream neurons constrain downstream ones. We expect $\rho \ll 1$ for realistic SNNs, and ρ to decrease as the network grows. This is what makes PRISM’s BFS reachability analysis effective — but the BDD representation still requires memory proportional to (a compression of) $|S_{\text{theory}}|$.

3.3. Chain Topology Analysis

Consider a chain of N processing neurons: $\text{In} \rightarrow n_1 \rightarrow n_2 \rightarrow \dots \rightarrow n_N$.

This topology has:

- $|V_{\text{in}}| = 1, |V_{\text{proc}}| = N$
- $|E| = N$ (one input edge + $N - 1$ internal edges)
- $|E_{\text{int}}| = N - 1$ (internal edges only)

For the **fast precise** model with $P_{\text{rth}} = 100$ and weight $w = 80$:

- Fan-in of each neuron is 1, so $R_n = P_{\text{max}} - P_{\text{min}} + 1$
- Per-neuron narrowing gives $P_{\text{max}} = \max(P_{\text{rth}}, 3w/2) = \max(100, 120) = 120$
- No inhibitory synapses, so $P_{\text{min}} = 0$
- Therefore $R_n = 121$ for each neuron

$$|S_{\text{theory}}^{\text{chain}}| = (T_{\text{max}} + 1) \cdot 2 \cdot \prod_{n=1}^N (2 \cdot 121) \cdot 2^{N-1}$$

With $T_{\text{max}} = 20$:

$$|S_{\text{theory}}^{\text{chain}}| = 21 \cdot 2 \cdot 242^N \cdot 2^{N-1} = 21 \cdot 2^N \cdot 242^N = 21 \cdot 484^N$$

For the **discretized** model with $W = 3$, $T_d = \lceil 100 \cdot \frac{3}{100} \rceil = 3$:

- $P_{\text{max}}^d = 3 + 3 = 6$ (max excitatory input is $\delta_3(80) = 3$)
- Per-neuron states: $2 \cdot 7 = 14$

$$|S_{\text{theory}}^{\text{chain, disc}}| = 21 \cdot 2 \cdot 14^N \cdot 2^{N-1} = 21 \cdot 28^N$$

Remark. The per-neuron factor drops from 484 (precise) to 28 (discretized $W = 3$), a **17.3× reduction per neuron**. For $N = 4$ neurons, this compounds to $17.3^4 \approx 89,500 \times$ overall.

4. Empirical Validation

We generated 63 PRISM models across 7 topologies, 3 model configurations (deterministic, fast, full), and 3 model types (precise, discretized $W=2$, discretized $W=3$). PRISM 4.9 was used to build each model and export the reachable state space via `-exporttrans` and `-exportstates`.

4.1. Results Summary

Topology	Precise (Det.)	Precise (Fast)	Precise (Full)	Disc. W=3 (Fast)
Single (1N)	5	6	18	6
Chain-2 (2N)	12	23	246	11
Chain-3 (3N)	23	275	13,180	28
Chain-4 (4N)	42	6,917	362,289	88
Fork (3N)	12	83	3,388	15
Diamond (5N)	15	1,511	OOM	98
Convergent (1N, 2in)	17	20	65	16

Table 2: Reachable states $|S_{\text{reachable}}|$ for selected configurations. All values from PRISM DTMC exports. “OOM” indicates CUDD BDD out-of-memory.

Topology	Precise (Det.)	Precise (Fast)	Precise (Full)	Disc. W=3 (Fast)
Single (1N)	5	7	23	9
Chain-2 (2N)	12	35	369	19
Chain-3 (3N)	23	569	27,080	57
Chain-4 (4N)	42	19,656	885,160	215
Fork (3N)	12	167	6,015	29
Diamond (5N)	15	3,603	—	378
Convergent (1N, 2in)	26	36	128	40

Table 3: Transitions $|T|$ for selected configurations.

4.2. Scaling Analysis

The empirical data reveals clear patterns:

4.2.1. Exponential Growth in Chain Length

For the **precise fast** model, the chain sequence grows:

$$6 \rightarrow 23 \rightarrow 275 \rightarrow 6917$$

The per-neuron multiplicative factor is:

$$r_{\text{fast}} \approx \frac{23}{6} \approx 3.8, \quad \frac{275}{23} \approx 12.0, \quad \frac{6917}{275} \approx 25.2$$

This acceleration occurs because each additional neuron’s *reachable* potential range grows with the cumulative input. The first neuron sees only input $x \in \{0, 1\}$ weighted by 80, giving potential values $\{0, 80, 120\}$ (with leak). Subsequent neurons see richer input distributions.

For the **precise full** model:

$$18 \rightarrow 246 \rightarrow 13,180 \rightarrow 362,289$$

$$r_{\text{full}} \approx 13.7, \quad 53.6, \quad 27.5$$

The refractory state machine ($s_n \times \text{aref}_n \times \text{rref}_n$) adds a multiplicative factor of up to $3 \times 3 \times 5 = 45$ per neuron.

4.2.2. Discretization Effect

Proposition. The discretized model achieves an **empirical reduction ratio** of:

$$R_{\text{disc}} = \frac{|S_{\text{reachable}}^{\text{precise}}|}{|S_{\text{reachable}}^{\text{disc}}|}$$

For chain topologies with fast config and $W = 3$:

Topology	$ S_{\text{precise}} $	$ S_{\text{disc}(W=3)} $	R_{disc}
Chain-2	23	11	$2.1 \times$
Chain-3	275	28	$9.8 \times$
Chain-4	6,917	88	$78.6 \times$

Table 4: State space reduction from discretization (fast config).

The reduction is *super-linear* in chain length because the per-neuron potential domain shrinks from $R_n \approx 121$ to $P_{\text{max}}^d + 1 \approx 7$, and the multiplicative compounding across neurons amplifies this exponentially.

4.2.3. Topology Impact: Fan-Out vs Fan-In

Comparing fork (fan-out) and convergent (fan-in) topologies:

- **Fork** ($1 \rightarrow N1, N1 \rightarrow \{N2, N3\}$): 83 states (fast precise) — the fan-out creates independent branches whose states multiply.
- **Convergent** ($\text{In1}, \text{In2} \rightarrow N1$): 20 states (fast precise) — multiple inputs to a single neuron increase its potential range but don't multiply modules.
- **Diamond** (fan-out then fan-in): 1,511 states (fast precise) — combines both effects. The convergent neuron $N4$ receives input from two neurons with independent state histories, creating a rich reachable state set.

Remark. Fan-out is more expensive than fan-in. Adding a parallel branch doubles the module count (neuron + transfer), while adding a convergent input only increases the potential range of the target neuron. This has important implications for network design: *wide* networks (many parallel paths) are exponentially harder to verify than *deep* networks (long chains) of the same total neuron count.

5. Memory Consumption Model

5.1. PRISM Memory Architecture

PRISM uses BDDs (Binary Decision Diagrams) or MTBDDs for symbolic model representation. The memory consumed depends on:

1. **BDD node count:** Proportional to (a compressed representation of) the state space
2. **Transition matrix storage:** Sparse encoding of $|T|$ transitions
3. **Java heap overhead:** JVM object headers, hash tables, etc.

For the explicit engine, memory scales roughly as:

$$M_{\text{explicit}} \approx |S_{\text{reachable}}| \cdot c_s + |T| \cdot c_t$$

where $c_s \approx 40$ bytes per state (index, variable values, hash) and $c_t \approx 24$ bytes per transition (source, destination, probability).

5.2. Empirical Memory Estimates

Using the explicit model estimates with $c_s = 40, c_t = 24$ bytes:

Model	States	Transitions	Est. Memory	Actual Outcome
Chain-4 fast precise	6,917	19,656	748 KB	OK
Chain-4 full precise	362,289	885,160	35.7 MB	OK (46s)
Diamond full precise	>1M est.	—	>64 MB est.	OOM (CUDD)
Chain-4 fast disc W=3	88	215	8.7 KB	OK (<1s)

Table 5: Memory estimates for selected models.

Remark. The diamond-full-precise model failed not on Java heap but on CUDD internal BDD memory. CUDD’s default memory limit is lower than the Java heap (typically 1 GB). This suggests that the BDD representation of the diamond topology is particularly unfriendly to BDD ordering heuristics — the crossing paths between $N2 \rightarrow N4$ and $N3 \rightarrow N4$ create variable interdependencies that resist BDD compression. The `-cuddmaxmem` flag can increase this limit, but the underlying exponential scaling remains.

5.3. Predictive Formula

Definition. The **state space predictor** for an SNN graph G with configuration C is:

$$\hat{S}(G, C) = |S_{\text{clock}}| \cdot 2^{|V_{\text{in}}|} \cdot \prod_{n \in V_{\text{proc}}} f_{n(C)} \cdot 2^{|E_{\text{int}}|}$$

where $f_{n(C)}$ is the per-neuron state count:

$$f_{n(C)} = \begin{cases} 2 \cdot R_{n(G)} & \text{precise fast} \\ 90 \cdot R_{n(G)} & \text{precise full (with ARP=2, RRP=4)} \\ 2 \cdot (T_d + E_n^d + 1) & \text{discretized} \end{cases}$$

with $R_{n(G)} = \max\left(P_{\text{rth}}, \frac{3}{2} \sum_{e \in \text{In}(n), w_e > 0} w_e\right) + 1$ (for excitatory-only fan-in) and $E_n^d = \sum_{e \in \text{In}(n), w_e^d > 0} w_e^d$.

The predictor $\hat{S}$ overestimates the reachable state space (since not all theoretical states are reachable), but provides a **conservative upper bound** for memory planning:

$$\hat{M}(G, C) = \hat{S}(G, C) \cdot (c_s + |(d)| \cdot c_t)$$

where $|(d)|$ is the average out-degree of the DTMC (typically ≈ 2 – 3 for CogSpike models due to probabilistic branching at threshold levels).

6. Impact of Weight Discretization on Scaling

The weight discretization scheme established in [1] replaces each neuron’s continuous potential domain with a smaller discretized domain. The key mechanism is the weight discretization function $\delta_{W(w)} = \lfloor w \cdot W / w_{\max} \rfloor$, which maps the original weight range $[-100, 100]$ to $[-W, W]$, and the calibrated threshold $T_d = \lceil T \cdot W / w_{\max} \rceil$, which ensures threshold preservation. The *weighted contribution* for each neuron becomes $C_n = \sum_i w_i^d \cdot y_i$, and the maximum potential is bounded by $T_d + E_n^d$ where E_n^d is the maximum excitatory contribution.

Critically, the Soundness Theorem [1] guarantees that this domain reduction does not introduce spurious spikes: the threshold-dependent leak factor $\lambda_d = -\max(1, \lfloor \ell \cdot T_d \rfloor)$ ensures potentials decay correctly, and the zero-potential-maps-to-zero-firing-probability property provides a complete safety chain from sub-threshold potential through decay to permanent silence.

We now characterize the *scaling impact* of this domain reduction when compounded across multiple neurons in a network topology.

Theorem (Exponential Reduction). For a chain of N neurons with $|V_{\text{in}}| = 1, |E_{\text{int}}| = N - 1$, the state space ratio between precise and discretized models is:

$$\frac{|S^{\text{precise}}|}{|S^{\text{disc}}|} = \prod_{n=1}^N \frac{R_n}{P_{\max,n}^d + 1}$$

For uniform fan-in = 1, weight w , and $W = 3$:

$$R_n \approx \max(P_{\text{rth}}, 3w/2) + 1 \quad \text{vs.} \quad P_{\max,n}^d + 1 = T_d + \delta_{W(w)} + 1$$

With $P_{\text{rth}} = 100, w = 80$: $R_n = 121$ vs. $P_{\max}^d + 1 = 7$.

Reduction per neuron: $\frac{121}{7} \approx 17.3 \times$

Reduction for N neurons: $\left(\frac{121}{7}\right)^N \approx 17.3^N$

For $N = 4$: $17.3^4 \approx 89,547 \times$

This exponential compounding explains why the discretized chain-4 model (88 states) is dramatically smaller than the precise version (6,917 states for fast, 362,289 for full). The soundness guarantee from [1] ensures this reduction is *safe*: no behavioral properties are lost.

7. Conclusions and Recommendations

7.1. Key Findings

1. **State space grows exponentially** in the number of processing neurons, with the base depending on model configuration (fast: 12–25 $\times$ per neuron, full: 27–54 $\times$).
2. **The weight discretization of [1] provides exponential reduction** when compounded across neurons. The δ_W mapping reduces each neuron’s potential domain from $O(P_{\text{rth}})$ to $O(W)$, with the Soundness Theorem guaranteeing safety. For $W = 3$, the per-neuron reduction is 17 $\times$, and biological properties (tonic spiking, integrator behavior, excitability) are preserved.
3. **Fan-out is more expensive than fan-in** for verification. Parallel branches multiply module counts, while convergent inputs only increase potential ranges.

4. **BDD memory (CUDD) is the practical bottleneck**, not Java heap. The BDD representation of diamond topologies is particularly problematic due to variable ordering challenges.
5. **The deterministic configuration** ($k = 1$, no ARP/RRP) provides the smallest state spaces, suitable for rapid feasibility checks before full verification.

7.2. Practical Limits

Based on our empirical data and the 2 GB CUDD default limit:

Configuration	Chain Max	Diamond Max	Notes
Det. precise	>10 neurons	>10 neurons	Minimal state space
Fast precise	5–6 neurons	4 neurons	With per-neuron narrowing
Full precise	4 neurons	<4 neurons	ARP/RRP dominate
Fast disc $W=3$	>10 neurons	7–8 neurons	Dramatically reduced
Full disc $W=3$	8 neurons	5–6 neurons	Best precision/tractability

Table 6: Estimated maximum network sizes for different configurations (2 GB CUDD limit).

7.3. Recommendations for CogSpike

1. **Pre-flight estimator:** Implement $\hat{S}(G, C)$ in CogSpike to warn users before launching PRISM when the predicted state space exceeds a configurable threshold.
2. **Engine selection:** Use PRISM’s explicit engine (`-e`) for models with >100K states; the BDD engine struggles with large, irregularly-structured models.
3. **CUDD memory flag:** Expose `-cuddmaxmem` as a configurable parameter in CogSpike’s PRISM options (currently only `-javamaxmem` is configurable).
4. **Hybrid verification strategy:** For large networks, verify subgraphs independently and compose results, exploiting the modular structure of the PRISM model.
5. **Progressive refinement:** Start with deterministic config, then fast, then full — using the cheaper configurations to establish baseline properties before investing in the exponentially more expensive full verification.

Bibliography

- [1] C. R. Team, “Weight Discretization for Quotient Model Abstraction,” 2026.
- [2] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.